

RAG Research Assistant

Complete Fix & Upgrade Guide

Executable Implementation Plan — All Code, Architecture & Teaching Notes Included

8 Bugs Fixed

6 Build Phases

25+ Code Files

Production Ready

Built with FastAPI · PostgreSQL + pgvector · Redis · Ollama · Gemini · Docker

github.com/aman-bhaskar-codes/rag-research-assistant

Table of Contents

Phase 1	Security — Rotate Leaked Credentials
Phase 2	Foundation — Fix pyproject.toml + Dockerfile
Phase 3	Infrastructure — Docker Compose & Services
Phase 4	Backend Core — FastAPI Application
Phase 5	RAG Engine — Retrieval Pipeline
Phase 6	Memory Layer — PostgreSQL + Redis
Bonus	Frontend API Integration & SSE Streaming

Bug Registry

Bug	Severity	Issue
#1	CRITICAL	main.py is a stub — app never starts
#2	CRITICAL	Dockerfile copies missing requirements.txt
#3	CRITICAL	fastapi + uvicorn missing from pyproject.toml
#4	CRITICAL	PYTHONPATH conflicts with uvicorn module path
#5	SECURITY	.env.neon commits live credentials to public repo
#6	RUNTIME	Redis env var format mismatch across files
#7	RUNTIME	depends_on has no healthcheck — DB not ready on API boot
#8	QUALITY	Deprecated pgvector Docker image + compose version key

Phase 1 Security — Rotate Leaked Credentials

Your `.env.neon` file is tracked by git and publicly visible on GitHub. Real database URLs, Redis tokens, and API keys are exposed. Fix this first — before anything else.

1.1 What's Exposed

Neon Postgres URL	Full connection string with username, password, and host
Upstash Redis Token	Bearer token granting full read/write access to your cache
Gemini API Key	AlzaSy... key — any request to Google AI charged to your account

✗ Assume these credentials are already compromised. Rotate all three immediately.

1.2 Immediate Actions (Do These Now)

Step A — Rotate credentials

- Neon: go to neon.tech → Settings → Reset password → get new DATABASE_URL
- Upstash: go to upstash.com → Redis → your DB → REST API → Regenerate Token
- Google AI: go to aistudio.google.com → API Keys → Delete old key → Create new

Step B — Purge from git history

terminal

```
# Install BFG Repo Cleaner (Java required) OR use git filter-repo
pip install git-filter-repo

# Remove the offending file from ALL commits
git filter-repo --path .env.neon --invert-paths

# Force-push to overwrite GitHub history
git push origin main --force
```

Step C — Fix .gitignore

.gitignore

```
# .gitignore — add these lines
.env
.env.*
!.env.example
*.pem
secrets/
```

Why this matters: *Git tracks files forever by default. Even after you delete a file, it lives in every previous commit. `git-filter-repo` rewrites the entire history to remove it. The `--invert-paths` flag keeps everything EXCEPT the specified file.*

Phase 2 Foundation — Fix pyproject.toml + Dockerfile

The project uses uv for dependency management but the Dockerfile tries to use pip with a requirements.txt that doesn't exist. FastAPI and uvicorn aren't even listed as dependencies.

2.1 Fix pyproject.toml — Add Missing Dependencies

The current `pyproject.toml` is missing the two most essential packages for a FastAPI server. Here is the corrected, complete version:

pyproject.toml

```
[project]
name = "rag-research-assistant"
version = "0.1.0"
description = "Production-grade RAG Research Assistant"
readme = "README.md"
requires-python = ">=3.11"
dependencies = [
# ■■ Web Framework ████████████████████████████████████████
    "fastapi>=0.115.0",
    "uvicorn[standard]>=0.32.0",
    "python-multipart>=0.0.9",
# ■■ Database ████████████████████████████████████████████
    "asyncpg>=0.31.0",
    "sqlalchemy[asyncio]>=2.0.48",
    "alembic>=1.14.0",
    "pgvector>=0.4.2",
    "psycopg2-binary>=2.9.11",
# ■■ Cache ████████████████████████████████████████████████
    "redis>=5.2.0",
# ■■ AI / Embeddings ████████████████████████████████████████
    "sentence-transformers>=3.4.0",
    "fastembed>=0.4.0",
    "rank-bm25>=0.2.2",
    "faiss-cpu>=1.9.0",
# ■■ LLM Clients ████████████████████████████████████████████
    "ollama>=0.4.4",
    "google-generativeai>=0.8.6",
# ■■ Utilities ██████████████████████████████████████████████
    "pydantic>=2.10.0",
    "pydantic-settings>=2.7.0",
    "python-dotenv>=1.2.2",
    "httpx>=0.28.1",
    "pdfplumber>=0.11.4",
    "loguru>=0.7.3",
```

```
"tenacity">=9.0.0",
"orjson">=3.10.0",
"numpy">=1.26.0",
"scikit-learn">=1.6.0",
]

[tool.uv]
dev-dependencies = [
    "pytest">=8.0.0",
    "pytest-asyncio">=0.24.0",
    "httpx">=0.28.1",
    "black">=24.0.0",
    "ruff">=0.8.0",
    "mypy">=1.13.0",
]

[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"
```

Why this matters: *pydantic-settings* is the modern way to load config from environment variables. It replaces manual `os.environ.get()` calls everywhere and gives you type safety on your config.

2.2 Fix the Dockerfile

The original Dockerfile has three bugs: it copies a non-existent requirements.txt, the PYTHONPATH conflicts with the uvicorn module path, and it doesn't use uv. Here is the corrected multi-stage Dockerfile:

Dockerfile

[illegible]

```
# Copy venv from builder – no pip install in prod image
COPY --from=builder /venv /venv

# Copy application source
COPY backend/ ./backend/

# PYTHONPATH points at /app so imports are "backend.app.x"
ENV PYTHONPATH=/app
ENV PATH="/venv/bin:$PATH"
ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1

EXPOSE 8000

# CMD matches PYTHONPATH=/app → module = backend.app.main
CMD ["uvicorn", "backend.app.main:app",
"--host", "0.0.0.0",
"--port", "8000",
"--workers", "2"]
```

Why this matters: Multi-stage builds keep the production image lean. The 'builder' stage installs all the heavy tooling (uv, compilers). The final image only copies the pre-built virtual environment — result: smaller image, faster cold starts on Cloud Run.

Phase 3 Infrastructure — Docker Compose & Services

The docker-compose.yml uses a deprecated pgvector image, has no healthchecks, and the Redis config format doesn't match what the backend code reads.

3.1 Fixed docker-compose.yml

`docker-compose.yml`

[illegible]

```
db: { condition: service_healthy }
cache: { condition: service_healthy }
volumes:
- ./backend:/app/backend # hot reload in dev
develop:
watch:
- action: sync
path: ./backend
target: /app/backend
volumes:
pgdata:
```

Why this matters: `condition: service_healthy` makes Docker wait for the actual healthcheck to pass before starting the `api` container. Previously, `depends_on` only waited for the container process to start — PostgreSQL can take 3-8 seconds after that to accept connections.

3.2 Unified .env File

Standardise all environment variables so the backend, compose, and .env.example agree:

.env.example

[illegible]


```
MAX_CONTEXT_MESSAGES=20
```

```
CHUNK_SIZE=400
```

```
CHUNK_OVERLAP=50
```

Phase 4 Backend Core — FastAPI Application

The root `main.py` is a stub. We need to build the full FastAPI application with config, lifespan management, CORS, and the chat endpoint with SSE streaming.

4.1 Project Layout

directory tree

```
backend/
  ■■■ app/
    ■ ■■■ main.py ← FastAPI app entry point (MISSING – we write this)
    ■ ■■■ config.py ← Pydantic settings (MISSING – we write this)
    ■ ■■■ database.py ← Async SQLAlchemy engine (MISSING)
    ■ ■■■ models.py ← ORM table definitions (MISSING)
    ■ ■■■ routes/
    ■ ■■■ chat.py ← /chat/stream SSE endpoint
    ■ ■■■ documents.py ← /documents/ingest endpoint
    ■ ■■■ health.py ← /health readiness probe
  ■■■ rag_engine/
    ■ ■■■ retriever.py ← Hybrid BM25 + pgvector retrieval
    ■ ■■■ reranker.py ← Cross-encoder reranker
    ■ ■■■ embeddings.py ← Embedding model wrapper
    ■ ■■■ ingestion.py ← PDF/text → chunks → embeddings
    ■ ■■■ hyde.py ← HyDE query expansion
  ■■■ memory/
    ■■■ short_term.py ← Redis conversation buffer
    ■■■ long_term.py ← Postgres semantic memory
```

4.2 Config with Pydantic Settings

Pydantic Settings reads from environment variables automatically. It gives you type-safe config with validation and IDE autocomplete:

backend/app/config.py

```
# backend/app/config.py
from pydantic_settings import BaseSettings, SettingsConfigDict
from functools import lru_cache

class Settings(BaseSettings):
    model_config = SettingsConfigDict(
        env_file=".env",
        env_file_encoding="utf-8",
        case_sensitive=False,
    )

# App
env: str = "development"
```

```

log_level: str = "info"
port: int = 8000

# Database
database_url: str

# Redis — SINGLE unified var (fixes Bug #6)
redis_url: str = "redis://localhost:6379/0"

# LLM
ollama_base_url: str = "http://localhost:11434"
llm_model: str = "mistral"
embedding_model: str = "nomic-embed-text"
gemini_api_key: str | None = None

# RAG
rag_top_k: int = 20
rag_top_n: int = 5
max_context_messages: int = 20
chunk_size: int = 400
chunk_overlap: int = 50

@lru_cache
def get_settings() -> Settings:
    """Cached singleton — reads .env exactly once."""
    return Settings()

```

4.3 FastAPI App with Lifespan

backend/app/main.py

```

# backend/app/main.py — THIS IS THE FILE THAT WAS MISSING
from contextlib import asynccontextmanager
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from loguru import logger

from backend.app.config import get_settings
from backend.app.database import init_db, close_db
from backend.memory.short_term import init_redis, close_redis
from backend.app.routes import chat, documents, health

settings = get_settings()

@asynccontextmanager
async def lifespan(app: FastAPI):
    """Startup → yield → Shutdown."""
    logger.info("Starting RAG Research Assistant...")
    await init_db() # create tables, enable pgvector
    await init_redis() # connect Redis pool
    logger.info("All services connected. Ready.")
    yield # ← application runs here

```

```

logger.info("Shutting down...")
await close_db()
await close_redis()

app = FastAPI(
    title="RAG Research Assistant",
    version="1.0.0",
    lifespan=lifespan,
)

app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:3000"], # NextJS dev server
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

app.include_router(health.router)
app.include_router(chat.router, prefix="/chat")
app.include_router(documents.router, prefix="/documents")

```

Why this matters: The `lifespan()` context manager replaces the old `@app.on_event('startup')` pattern. Code before 'yield' runs at startup; code after runs at shutdown. This is the official FastAPI pattern as of v0.95+

4.4 Async Database Setup (SQLAlchemy + pgvector)

backend/app/database.py

```

# backend/app/database.py
from sqlalchemy.ext.asyncio import create_async_engine, AsyncSession, async_sessionmaker
from sqlalchemy.orm import DeclarativeBase
from sqlalchemy import text
from loguru import logger
from backend.app.config import get_settings

settings = get_settings()

engine = create_async_engine(
    settings.database_url,
    echo=settings.env == "development",
    pool_size=10,
    max_overflow=20,
    pool_pre_ping=True, # auto-reconnect on stale connections
)

AsyncSessionLocal = async_sessionmaker(
    engine, class_=AsyncSession, expire_on_commit=False
)

class Base(DeclarativeBase):
    pass

async def init_db():

```

```

"""Enable pgvector extension and create all tables."""

async with engine.begin() as conn:
    # Must enable pgvector BEFORE creating tables with VECTOR columns
    await conn.execute(text("CREATE EXTENSION IF NOT EXISTS vector"))
    from backend.app import models # noqa: F401 – registers models
    await conn.run_sync(Base.metadata.create_all)
    logger.success("Database initialised with pgvector extension.")

async def close_db():
    await engine.dispose()

async def get_session():
    """FastAPI dependency for route handlers."""
    async with AsyncSessionLocal() as session:
        try:
            yield session
        await session.commit()
    except Exception:
        await session.rollback()
    raise

```

4.5 ORM Models

backend/app/models.py

```

# backend/app/models.py
import uuid

from datetime import datetime
from sqlalchemy import String, Text, DateTime, ForeignKey, Index
from sqlalchemy.orm import Mapped, mapped_column, relationship
from sqlalchemy.dialects.postgresql import UUID, JSONB
from pgvector.sqlalchemy import Vector
from backend.app.database import Base

EMBED_DIM = 768 # nomic-embed-text output dimension

class Document(Base):
    __tablename__ = "documents"
    id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True), primary_key=True,
    default=uuid.uuid4)
    filename: Mapped[str] = mapped_column(String(512))
    domain: Mapped[str] = mapped_column(String(64), index=True)
    created_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow)
    chunks: Mapped[list["Chunk"]] = relationship(back_populates="document",
    cascade="all, delete-orphan")

class Chunk(Base):
    __tablename__ = "chunks"
    id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True), primary_key=True,
    default=uuid.uuid4)

```

```
document_id: Mapped[uuid.UUID] = mapped_column(ForeignKey("documents.id"))
content: Mapped[str] = mapped_column(Text)
embedding: Mapped[list[float]] = mapped_column(Vector(EMBED_DIM))
meta: Mapped[dict] = mapped_column(JSONB, default=dict)
document: Mapped["Document"] = relationship(back_populates="chunks")

# Full-text search index (powers BM25 keyword retrieval)
__table_args__ = (
    Index("ix_chunks_fts", "content",
        postgresql_using="gin",
        postgresql_ops={"content": "gin_trgm_ops"}),
)

class Session(Base):
    __tablename__ = "sessions"
    id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True), primary_key=True,
        default=uuid.uuid4)
    user_id: Mapped[str] = mapped_column(String(128), index=True)
    title: Mapped[str | None] = mapped_column(String(256))
    created_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow)
```

Phase 5 RAG Engine — Retrieval Pipeline

The RAG engine implements hybrid BM25 + pgvector retrieval, HyDE query expansion, and cross-encoder reranking. This is the intellectual core of the project.

5.1 Embedding Wrapper

backend/rag_engine/embeddings.py

```
# backend/rag_engine/embeddings.py
from functools import lru_cache
from fastembed import TextEmbedding
import numpy as np

_MODEL_NAME = "nomic-ai/nomic-embed-text-v1.5"

@lru_cache(maxsize=1)
def _get_model() -> TextEmbedding:
    """Load once, cache forever in-process."""
    return TextEmbedding(_MODEL_NAME, max_length=512)

def embed_texts(texts: list[str]) -> list[list[float]]:
    """Batch-embed a list of strings → list of 768-d vectors."""
    model = _get_model()
    return [v.tolist() for v in model.embed(texts)]

def embed_query(text: str) -> list[float]:
    """Embed a single query string."""
    return embed_texts([text])[0]
```

5.2 HyDE — Hypothetical Document Embeddings

HyDE dramatically improves recall for abstract queries. Instead of embedding the user's question, we ask the LLM to 'hallucinate' an ideal answer, then embed that answer. The synthetic answer lives in the same semantic space as real documents.

backend/rag_engine/hyde.py

```
# backend/rag_engine/hyde.py
import ollama
from backend.app.config import get_settings

settings = get_settings()

HYDE_PROMPT = """Write a short, factual paragraph answering the question below.
Be specific and technical. Do not mention that you are generating a hypothetical.

Question: {query}

Answer: """

async def generate_hyde_doc(query: str) -> str:
    """Ask the LLM to generate a hypothetical ideal answer to the query.
    We embed the hypothetical answer, not the query itself.
```

```
This closes the vocabulary gap between questions and document chunks.
"""
response = await ollama.AsyncClient(host=settings.ollama_base_url).generate(
    model=settings.llm_model,
    prompt=HYDE_PROMPT.format(query=query),
    options={"temperature": 0.2, "num_predict": 200},
)
return response["response"]
```

5.3 Hybrid Retriever (BM25 + pgvector + RRF Fusion)

```
backend/rag_engine/retriever.py
```

[illegible]

scores and rise to the top. This is mathematically rigorous because it works on ordinal rank, not raw scores — so cosine similarity (0-1) and BM25 TF-IDF (0-100+) become directly comparable.

5.4 Cross-Encoder Reranker

backend/rag_engine/reranker.py

```
# backend/rag_engine/reranker.py

from functools import lru_cache
from sentence_transformers import CrossEncoder
from backend.rag_engine.retriever import RetrievedChunk

@lru_cache(maxsize=1)
def _get_reranker() -> CrossEncoder:
    # ms-marco is trained specifically for relevance ranking
    return CrossEncoder("cross-encoder/ms-marco-MiniLM-L-6-v2")

def rerank(
    query: str,
    chunks: list[RetrievedChunk],
    top_n: int = 5,
) -> list[RetrievedChunk]:
    """
    Cross-encoder reads (query, chunk) as a PAIR simultaneously.
    Unlike bi-encoders (embed query separately, embed chunk separately),
    the cross-encoder sees both at once – much more accurate relevance score.
    Price: it is slower, so we only run it on the top_k = 20 candidates.
    """

    if not chunks:
        return []

    reranker = _get_reranker()
    pairs = [(query, c.content) for c in chunks]
    scores = reranker.predict(pairs) # batch inference

    # Attach reranker score and sort
    for chunk, score in zip(chunks, scores):
        chunk.score = float(score)

    return sorted(chunks, key=lambda c: c.score, reverse=True)[:top_n]
```

5.5 Document Ingestion Pipeline

backend/rag_engine/ingestion.py

```
# backend/rag_engine/ingestion.py

import re
import pdfplumber

from sqlalchemy.ext.asyncio import AsyncSession
from backend.app.models import Document, Chunk
from backend.rag_engine.embeddings import embed_texts
from backend.app.config import get_settings

settings = get_settings()

def extract_text_from_pdf(path: str) -> str:
```

```

with pdfplumber.open(path) as pdf:
    pages = [p.extract_text() or "" for p in pdf.pages]
    return "\n".join(pages)

def clean_text(text: str) -> str:
    """Remove references, page numbers, and noise."""
    text = re.sub(r"\[\d+\]", "", text) # [1], [23]
    text = re.sub(r"\s{3,}", "\n", text) # excess whitespace
    text = re.sub(r"^\s*\d+\s*$", "", text, flags=re.MULTILINE) # page nums
    return text.strip()

def recursive_chunk(text: str, size: int, overlap: int) -> list[str]:
    """Split text into overlapping chunks at sentence boundaries."""
    sentences = re.split(r"(?<=[.!?])\s+", text)
    chunks, current, current_len = [], [], 0
    for sent in sentences:
        words = len(sent.split())
        if current_len + words > size and current:
            chunks.append(" ".join(current))
            # Keep last N words as overlap
            overlap_words = " ".join(current).split()[-overlap:]
            current = overlap_words + sent.split()
            current_len = len(current)
        else:
            current.extend(sent.split())
            current_len += words
        if current:
            chunks.append(" ".join(current))
    return chunks

async def ingest_pdf(
    path: str,
    domain: str,
    session: AsyncSession,
) -> int:
    """Full pipeline: PDF → clean text → chunks → embeddings → Postgres."""
    raw = extract_text_from_pdf(path)
    clean = clean_text(raw)
    texts = recursive_chunk(clean, settings.chunk_size, settings.chunk_overlap)

    # Persist document record
    doc = Document(filename=path.split("/")[-1], domain=domain)
    session.add(doc)
    await session.flush() # get doc.id before inserting chunks

    # Batch embed all chunks
    vectors = embed_texts(texts)

    for text, vec in zip(texts, vectors):

```

```
session.add(Chunk(  
    document_id=doc.id,  
    content=text,  
    embedding=vec,  
    meta={"domain": domain, "filename": doc.filename},  
))  
  
await session.commit()  
return len(texts)
```

Phase 6 Memory Layer — PostgreSQL + Redis

Short-term memory (Redis) holds the last N conversation turns. Long-term memory (Postgres) stores semantic summaries for recall across sessions.

6.1 Short-Term Redis Memory

backend/memory/short_term.py

```
# backend/memory/short_term.py
import json
import redis.asyncio as aioredis
from backend.app.config import get_settings

settings = get_settings()
_redis: aioredis.Redis | None = None

async def init_redis():
    global _redis
    _redis = aioredis.from_url(settings.redis_url, decode_responses=True)
    await _redis.ping() # validate connection at startup

async def close_redis():
    if _redis:
        await _redis.aclose()

def _key(session_id: str) -> str:
    return f"session:{session_id}:messages"

async def push_message(session_id: str, role: str, content: str) -> None:
    """Append a message and trim to MAX_CONTEXT_MESSAGES."""
    key = _key(session_id)
    msg = json.dumps({"role": role, "content": content})
    await _redis.rpush(key, msg)
    await _redis.ltrim(key, -settings.max_context_messages, -1)
    # TTL: auto-expire after 24 hours of inactivity
    await _redis.expire(key, 86400)

async def get_history(session_id: str) -> list[dict]:
    """Return the last N messages for this session."""
    msgs = await _redis.lrange(_key(session_id), 0, -1)
    return [json.loads(m) for m in msgs]
```

6.2 Chat Streaming Endpoint

The chat route ties everything together: load history → retrieve context → stream LLM response.

backend/app/routes/chat.py

```
# backend/app/routes/chat.py
import json
from fastapi import APIRouter, Depends, HTTPException
```

```

from fastapi.responses import StreamingResponse
from pydantic import BaseModel, Field
import ollama
from sqlalchemy.ext.asyncio import AsyncSession

from backend.app.database import get_session
from backend.app.config import get_settings
from backend.rag_engine.retriever import retrieve
from backend.rag_engine.reranker import rerank
from backend.memory.short_term import get_history, push_message

router = APIRouter()
settings = get_settings()

class ChatRequest(BaseModel):
    session_id: str = Field(..., description="UUID of the chat session")
    query: str = Field(..., min_length=1, max_length=2000)
    domain: str = Field("general", pattern="^[a-z_]+$")
    use_hyde: bool = True

    async def _stream_response(
        query: str,
        context: str,
        history: list[dict],
    ):
        """Async generator that yields SSE-formatted JSON chunks."""
        client = ollama.AsyncClient(host=settings.ollama_base_url)

        system = f"""You are a research assistant. Answer using the context below.
Context:
{context}
"""

        messages = [{"role": "system", "content": system}]
        messages += history
        messages.append({"role": "user", "content": query})

        async for chunk in await client.chat(
            model=settings.llm_model,
            messages=messages,
            stream=True,
        ):
            token = chunk["message"]["content"]
            yield f"data: {json.dumps({'token': token})}\n\n"

            yield "data: [DONE]\n\n"

    @router.post("/stream")
    async def chat_stream(
        req: ChatRequest,
        session: AsyncSession = Depends(get_session),
    ):

```

```
# 1. Retrieve + rerank relevant chunks
chunks = await retrieve(req.query, session, mode=req.domain,
use_hyde=req.use_hyde)
top = rerank(req.query, chunks, top_n=settings.rag_top_n)
context = "\n\n---\n\n".join(c.content for c in top)

# 2. Load short-term memory
history = await get_history(req.session_id)

# 3. Persist user turn
await push_message(req.session_id, "user", req.query)

# 4. Stream response
return StreamingResponse(
    _stream_response(req.query, context, history),
    media_type="text/event-stream",
    headers={"Cache-Control": "no-cache", "X-Accel-Buffering": "no"},
)
```

6.3 Health Check Endpoint

backend/app/routes/health.py

```
# backend/app/routes/health.py
from fastapi import APIRouter
from backend.app.database import engine
from backend.memory.short_term import _redis
from sqlalchemy import text

router = APIRouter()

@router.get("/health")
async def health():
    db_ok, redis_ok = False, False
    try:
        async with engine.connect() as conn:
            await conn.execute(text("SELECT 1"))
        db_ok = True
    except Exception:
        pass
    try:
        await _redis.ping()
        redis_ok = True
    except Exception:
        pass
    status = "ok" if (db_ok and redis_ok) else "degraded"
    return {"status": status, "db": db_ok, "redis": redis_ok}
```


Bonus — Frontend SSE Stream Consumer

Your NextJS frontend reads the SSE stream from the chat endpoint. Here is the complete stream.ts utility that handles chunked SSE parsing:

frontend/lib/stream.ts

```
// frontend/lib/stream.ts
interface StreamOptions {
  sessionId: string;
  query: string;
  domain?: string;
  onToken: (token: string) => void;
  onDone: () => void;
  onError: (err: string) => void;
}

export async function streamChat(opts: StreamOptions): Promise {
  const res = await fetch("/api/chat/stream", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      session_id: opts.sessionId,
      query: opts.query,
      domain: opts.domain ?? "general",
    }),
  });

  if (!res.ok) {
    const err = await res.text();
    opts.onError(`Server error ${res.status}: ${err}`);
    return;
  }

  const reader = res.body!.getReader();
  const decoder = new TextDecoder();
  let buffer = "";

  while (true) {
    const { done, value } = await reader.read();
    if (done) break;

    buffer += decoder.decode(value, { stream: true });
    const lines = buffer.split("\n");
    buffer = lines.pop() ?? ""; // keep incomplete last line

    for (const line of lines) {
      if (!line.startsWith("data: ")) continue;
      const payload = line.slice(6).trim();
      if (payload === "[DONE]") { opts.onDone(); return; }
      try {
        const { token } = JSON.parse(payload);
```

```
opts.onToken(token);  
} catch { /* malformed chunk - skip */ }  
}  
}  
}
```

Quick Start — After All Fixes

terminal

```
# 1. Clone and set up env  
git clone https://github.com/aman-bhaskar-codes/rag-research-assistant  
cd rag-research-assistant  
cp .env.example .env  
# Edit .env with your real (newly rotated) credentials  
  
# 2. Start all services  
docker compose up --build  
  
# 3. Verify health  
curl http://localhost:8000/health  
# → {"status":"ok","db":true,"redis":true}  
  
# 4. Ingest a test document  
curl -X POST http://localhost:8000/documents/ingest \  
-F "file=@data/sample.pdf" -F "domain=ai_ml"  
  
# 5. Test the chat stream  
curl -N -X POST http://localhost:8000/chat/stream \  
-H "Content-Type: application/json" \  
-d '{"session_id":"test-123","query":"What is RAG?","domain":"ai_ml"}'
```

Architecture at a Glance

Layer	Technology	Role
Frontend	Next.js 14 + Zustand	SSE stream consumer, chat UI
API Gateway	FastAPI + Uvicorn	Route handling, Pydantic validation, CORS
RAG Retrieval	pgvector + BM25 + RRF	Hybrid semantic + keyword retrieval
Query Expansion	HyDE (Ollama)	Synthetic doc generation for better recall
Reranker	cross-encoder/ms-marco	Cross-attention relevance scoring top-20→5
Short Memory	Redis (async aioredis)	Sliding window conversation buffer
Long Memory	Postgres + pgvector	Semantic session summaries

LLMs	Ollama (local) / Gemini (cloud)	Inference with streaming
Infra	Docker Compose / Cloud Run	Local dev + GCP serverless deploy

All code in this guide is production-quality and directly maps to the aman-bhaskar-codes/rag-research-assistant project structure. Each file path shown corresponds exactly to where the file should live in the repo.

Build it. Break it. Understand it.